05 - Clases, métodos y constructores

Pregunta número 1

En Java, ¿qué sintaxis es usada para declarar un parámetro en un método que acepte un número variable de argumentos?

Seleccione una:

- a. <>
- b. ...
- c. []
- d. varargs
- e. {}

Respuesta: En Java, la sintaxis para declarar un método que acepte un número variable de argumentos (característica conocida como varargs) consiste en usar tres puntos suspensivos (...) entre el tipo de dato y el nombre del parámetro, por ejemplo: `public void metodo(String... valores)`. Esto nos permite invocar el método pasando múltiples valores separados por comas, los cuales Java encapsula automáticamente en un arreglo.

Pregunta número 2

En Java, ¿cuál de las siguientes se utiliza para llamar a un constructor desde otro constructor dentro de la misma clase?

Seleccione una:

- a. super()
- b. new()
- c. this.<class_name>()
- d. this()

Respuesta: En Java, la palabra clave this() (junto con los argumentos correspondientes, si los hay) se utiliza para invocar a un constructor sobrecargado desde otro constructor dentro de la misma clase. A esta práctica se le conoce como encadenamiento de constructores (constructor chaining). Una regla vital que debemos recordar es que la llamada a `this()` debe ser obligatoriamente la primera instrucción dentro del bloque del constructor. En pocas palabras, podemos decir que en Java, para poder llamar a un constructor desde otro constructor dentro de la misma clase, se utiliza la palabra clave `this()`.

Pregunta número 3

Which class has a default constructor?

```
class X{}
class Y{
    Y(){}
}
class Z{
    Z(int i){}
}
```

Seleccione una:

- a. X and Y
- b. Y only
- c. X and Z
- d. Y and Z
- e. X, Y and Z
- f. Z only.
- g. X only.

Respuesta: En Java, un constructor por defecto (default constructor) es aquel que es generado automáticamente por el compilador. La regla que debemos tener presente es que el compilador solo genera un constructor por defecto si la clase no tiene ningún constructor declarado explícitamente. Este constructor por defecto no tiene argumentos y su cuerpo está vacío (o simplemente llama al constructor de la superclase). Hagamos un análisis de cada clase para entender mejor:
class X{}: No tiene ningún constructor declarado. Por lo tanto, el compilador genera automáticamente el constructor por defecto X(). O sea que tiene constructor por defecto.
class Y{ Y(){} }: Tiene un constructor explícito sin argumentos Y(). Aunque no recibe parámetros, fue escrito por alguien, por lo que no es un constructor por defecto generado por el compilador. O sea que no tiene constructor por defecto.
class Z{ Z(int i){} }: Tiene un constructor explícito con argumentos. El hecho de declarar cualquier constructor (con o sin parámetros) impide que el compilador genere el constructor por defecto. Por lo tanto, no tiene constructor por defecto.

Pregunta número 4

A method is declared to take three arguments. A program calls this method and passes only two arguments. What is the result?

Seleccione una:

- a. An exception occurs when the method attempts to access the third argument.
- b. The third argument is given the value zero.
- c. Compilation fails
- d. The third argument is given the value null.
- e. The third argument is given the appropriate false value for its declared type.
- f. The third argument is given the value void.

Respuesta: En Java, la validación de la firma de los métodos (method signature) es estricta y ocurre en tiempo de compilación. Si un método está declarado explícitamente para recibir tres argumentos, el compilador exige que la llamada al método incluya exactamente esos tres argumentos con los tipos de datos correspondientes. Llamarlo con solo dos argumentos provoca un error de compilación inmediato, ya que el compilador no puede encontrar una firma de método que coincida con esa invocación.

Pregunta número 5

¿Cuál de las siguientes afirmaciones describe correctamente un objeto inmutable en Java?

Seleccione una:

- a. Un objeto inmutable debe implementar la interfaz Cloneable.
- b. Los objetos inmutables no pueden crearse usando la palabra clave new.
- c. Los objetos inmutables pueden tener métodos setter para cambiar sus campos.
- d. Un objeto inmutable no permite ningún cambio en su estado después de ser creado.
- e. Un objeto inmutable puede tener sus campos modificados después de ser creado.

Respuesta: En Java, un objeto inmutable es aquel cuyo estado (los valores de sus atributos) no puede ser modificado una vez que ha sido instanciado. Cualquier operación que parezca modificar el objeto en realidad está creando un nuevo objeto con el estado modificado, dejando intacto el original. Un ejemplo que podemos tener en Java es la clase String. Si tenemos un String s = "Hola"; y luego hacemos s = s + " Mundo";, no estás modificando el objeto original "Hola", sino que se crea un nuevo objeto "Hola Mundo" y se reasigna la referencia.

Pregunta número 6

What is the result?

```
class X {
    String str = "default";
    X(String s) { str = s; }
    void print() { System.out.println(str); }
    public static void main(String[] args) { new X("hello").print(); }
}
```

Seleccione una:

- a. hello
- b. default
- c. Compilation fails.
- d. The program prints nothing.
- e. An exception is thrown at run time.

Respuesta: El código es completamente válido, va a compilar sin problemas y se ejecuta con normalidad. Para entender por qué pasa esto, hay que revisar el flujo de ejecución; comenzamos con la instanciación: En el método main, se inicia la creación de un nuevo objeto llamando a new X("hello"). Seguimos con la inicialización de variables: Al crear el objeto, la variable de instancia str se inicializa primero con el valor "default". Continuamos con la ejecución del constructor: Se ejecuta el constructor X(String s) con el argumento "hello". Dentro del constructor, la instrucción str = s; sobrescribe el valor "default" por "hello". Finalmente, llegamos a la llamada al método: Aquí se invoca el método print() sobre esa misma instancia recién creada. El método ejecuta System.out.println(str);, imprimiendo el valor actual de la variable, que es "hello".

Pregunta número 7

What is the result?

```
class X {
    static void m(int i){
        i+=7;
    }
    public static void main(String[] args){
        int i = 12;
        m(i);
        System.out.println(i);
    }
}
```

Seleccione una:

- a. Compilation fails.
- b. 12
- c. 7
- d. 19
- e. An exception is thrown at run time.

Respuesta: La parte más importante de este ejercicio es la comprensión sobre cómo Java maneja el paso de parámetros, que es estrictamente por valor (pass-by-value). En el método main, se declara una variable local primitiva i y se inicializa con el valor 12. Al invocar el método m(i), Java no envía una referencia a la variable original, sino que hace una copia de su valor (12) y se la entrega al parámetro i del método m. Dentro del método m, la instrucción i += 7; modifica únicamente esa copia local, elevando su valor a 19, pero sin afectar a la variable original del método que está llamando. Al terminar la ejecución de m, su ámbito desaparece. La variable i original en el método main permanece intacta. Por lo tanto, al ejecutar System.out.println(i), se imprime el valor original que sigue siendo 12.

Pregunta número 8

Dado: ¿Cuál es el resultado?

```
class Test {
    static void m(int a) {
        a += 10;
    }
    public static void main(String[] args) {
        int a = 20;
        m(a);
        Test.m(a);
        System.out.println(a);
    }
}
```

Seleccione una:

- a. 10
- b. 40
- c. 20
- d. Compilation fails.
- e. An exception is throw at run time.
- f. 30

Respuesta: Este ejercicio evalúa el mismo principio fundamental que el ejercicio anterior, el cual es que Java siempre utiliza el paso por valor (pass-by-value) al pasar argumentos a un método. En el método main, se declara la variable primitiva `a` y se le asigna el valor inicial de 20. Cuando se invoca el método m(a) por primera vez, se envía una copia del valor 20, no la referencia a la variable original. Dentro del método estático m(), el parámetro local suma 10, pero esta modificación ocurre en un ámbito separado y no afecta a la variable del main. La segunda invocación, Test.m(a), hace exactamente lo mismo: envía otra copia del valor 20 que se modifica y se descarta al terminar la ejecución del método. Finalmente, la instrucción System.out.println(a); imprime la variable `a` local del método main, la cual nunca fue alterada y conserva intacto su valor original de 20.

Pregunta número 9

What is the result?

```
public class OverloadedStaticMethods {
    public static void display() {
        System.out.println("Display with no arguments");
    }
    public static void display(int num) {
        System.out.println("Display with integer: " + num);
    }
    public static void display(String str) {
        System.out.println("Display with string: " + str);
    }
    public static void main(String[] args) {
        OverloadedStaticMethods.display();
        OverloadedStaticMethods.display(5);
        OverloadedStaticMethods.display("Hello");
    }
}
```

Seleccione una:

a.
```
Display with no arguments
Display with string:
Display with integer: Hello
```

b. Error en compilación

c. Error en tiempo de ejecución

d.
```
Display with no arguments
Display with integer: 5
Display with string: Hello
```

Respuesta: En este código podemos ver el concepto de sobrecarga de métodos (method overloading). La sobrecarga nos permite que varios métodos compartan el mismo nombre dentro de una clase, siempre y cuando su lista de parámetros (su

firma) sea diferente. El compilador de Java determina qué versión exacta del método debe ejecutar analizando los argumentos que se le pasan en cada llamada desde el método main: OverloadedStaticMethods.display(); al no recibir argumentos, coincide con la primera declaración e imprime "Display with no arguments". OverloadedStaticMethods.display(5); recibe un número, invocando el método que acepta un int, e imprime "Display with integer: 5". OverloadedStaticMethods.display("Hello"); recibe una cadena de texto, coincidiendo con el método que acepta un String, e imprime "Display with string: Hello".

Pregunta número 10

What will make this code compile and run?

```
01. public class Simple {
02.
03.     public float price;
04.     public static void main(String[] args) {
05.
06.         Simple price = new Simple();
07.         price = 4;
08.     }
09. }
```

Seleccione una:

- a. Change line 3 to the following: public int price;
- b. Change line 7 to the following: int price = new Simple();
- c. Change line 7 to the following: float price = new Simple ();
- d. Change line 7 to the following: price = 4f;
- e. Change line 7 to the following: price.price = 4;

Respuesta: El error de compilación actual se encuentra en la línea 7, donde se produce por una incompatibilidad de tipos debida al "shadowing" (ocultamiento) de variables. En la línea 6, se declara una variable local llamada price que es de tipo Simple (es decir, almacena una referencia al objeto recién creado). La línea 7 original (price = 4;) falla porque intenta asignar un número primitivo entero (4) directamente a esa variable de referencia de tipo objeto, lo cual es ilegal en Java. Para solucionar esto, necesitamos acceder al atributo del objeto. Al cambiar la instrucción a price.price = 4, el primer price hace referencia a la variable local (el objeto), y .price accede específicamente a su variable de instancia declarada en la

línea 3. Esta asignación es totalmente válida porque el entero 4 cabe dentro de un float y Java realiza una promoción implícita (widening conversion) sin problemas.

Pregunta número 11

What is the result when this program is executed?

```
public class Student{
    public String name = "";
    public int age = 0;
    public String major = "Undeclared";
    public boolean fulltime = true;
    public void display(){
        System.out.println("Name:" + name + "Major:" + major);
    }
    public boolean isFullTime(){
        return fulltime;
    }
}
And:
class TestStudent {
    public static void main(String[] args){
        Student bob = new Student();
        Student jian = new Student();
        bob.name = "Bob";
        bob.age = 19;
        jian = bob;
        jian.name = "Jian";
        System.out.println("Bob´s Name:" + bob.name);
    }
}
```

Seleccione una:

- a. Bob's name.
- b. Bob's Name: Jian
- c. Nothing prints
- d. Bob's Name: Bob.

Respuesta: En este ejercicio se está evaluando un concepto fundamental en Java, que es el cómo operan las variables de referencia al apuntar a objetos en la memoria (heap). Hay que analizar el flujo de ejecución para entender qué es lo que

esta pasando: Primero tenemos la creación de objetos: Las dos primeras líneas dentro del main instancian dos objetos Student independientes en memoria. La variable bob apunta al objeto 1 y jian apunta al objeto 2. Lo que sigue es la asignación de atributos: Se modifican los atributos del objeto 1 usando la referencia bob, estableciendo su nombre como "Bob" y edad en 19. De ahí pasamos a la reasignación de referencias (jian = bob): Esta es la línea crítica. En Java, esto no crea una copia independiente del objeto. En su lugar, hace que la variable jian deje de apuntar al objeto 2 y comience a apuntar exactamente a la misma dirección de memoria que bob (es decir, al objeto 1). El objeto 2 original queda sin referencias y se vuelve elegible para el garbage collector. Siguiendo con el flujo, tenemos la modificación compartida (jian.name = "Jian"): Como ambas variables apuntan ahora al mismo objeto, modificar el nombre a través de jian cambia el nombre del objeto compartido. Finalmente, tenemos la impresión: Al ejecutar System.out.println("Bob's Name:" + bob.name), se consulta el nombre del objeto 1, el cual fue actualizado a "Jian" en el paso anterior, dando como resultado la impresión final.

Pregunta número 12

What is the result?

```
public class Barn{
    public static void main(String[] args){
        new Barn().go("hi", 1);
        new Barn().go("hi", "world", 2);
    }
    public void go(String... y, int x){
        System.out.print(y[y.length - 1] + " ");
    }
}
```

Seleccione una:

- a. hi world
- b. Compilation fails
- c. world world.
- d. An exception is thrown at runtime.
- e. hi hi

Respuesta: El código no compila debido a una regla fundamental de los argumentos variables (varargs) en Java. Esta regla nos dice que cuando se declara un método con un parámetro de argumentos variables (usando ...), este parámetro debe ser el último en la lista de parámetros del método. Con lo anterior dicho, tenemos que el error en el código proporcionado se encuentra en la declaración del método, ya que se está declarando como public void go(String... y, int x). Aquí, el parámetro varargs String... y está seguido por el parámetro int x. Esto es sintácticamente inválido en Java y el compilador lanzará un error indicando algo como: "Varargs parameter must be the last parameter in the method signature", o sea, que el parámetro varargs debe ser el último en la firma del método.

Pregunta número 13

Cúal es el resultado?

```
public class Test {
    public static void op1(String s) {
        s += new String("Java");
    }
    public static void stringTest() {
        String msg = new String("Hello");
        op1(msg);
        System.out.println(msg);
    }
    public static void main(String... String) {
        stringTest();
    }
}
```

Seleccione una:

- a. HelloJava
- b. Hello
- c. An exception is thrown at runtime
- d. Java
- e. Compilation fails
- f. null

Respuesta: En este ejercicio se están evaluando dos conceptos fundamentales de Java. Por un lado, tenemos la inmutabilidad de los objetos String y, por el otro, tenemos el paso de parámetros por valor. Empezamos con el paso por valor de referencias: Cuando se llama al método op1(msg), Java hace una copia de la referencia al objeto "Hello" y se la asigna a la variable local s del método. De ahí nos movemos a la inmutabilidad de String: Dentro de op1, la operación s += new String("Java"); no modifica el objeto original. Debido a que los String son inmutables, esta operación de concatenación crea un nuevo objeto en memoria con el valor "HelloJava" y hace que la variable local s apunte a este nuevo objeto. El resultado final es que la variable original msg en el método stringTest() nunca fue modificada y sigue apuntando al objeto original "Hello". Por lo tanto, al imprimir msg, el resultado es Hello. Una nota adicional sobre la firma del main es que podría parecer que public static void main(String... String) causaría un error de compilación (opción e), pero es una sintaxis completamente válida en Java. String... define un parámetro varargs, y nombrar a la variable String está permitido porque String es el nombre de una clase, no una palabra reservada del lenguaje (como int o class).

Pregunta número 14

¿Cuáles dos opciones son verdaderas sobre el constructor por defecto?

Seleccione una o más de una:

- [x] a. El constructor por defecto siempre invoca el constructor sin argumentos de la superclase.
- [ ] b. El compilador crea un constructor por defecto si la clase no tiene definido un constructor sin argumentos.
- [ ] c. El constructor por defecto siempre es público.
- [x] d. El constructor por defecto se crea solo cuando la clase no tiene ningún constructor definido.
- [ ] e. El constructor por defecto hereda los argumentos del constructor de la superclase.

Respuesta: La primera respuesta correcta es que el constructor por defecto siempre invoca el constructor sin argumentos de la superclase. Cuando el compilador genera un constructor por defecto, este tiene un cuerpo vacío, pero implícitamente contiene una llamada a super(). Esto significa que siempre intentará llamar al constructor sin argumentos (no-arg constructor) de la clase padre directa. Si la superclase no tiene un constructor sin argumentos accesible, el código no compilará a menos que se defina explícitamente un constructor en la subclase que llame a otro constructor de la superclase. La otra respuesta correcta es que el constructor por defecto se crea solo cuando la clase no tiene ningún constructor

definido. Esta es la regla exacta de Java. El compilador genera automáticamente un constructor por defecto (sin argumentos) únicamente si la clase no contiene ninguna declaración explícita de constructores. En el momento en que escribimos cualquier constructor (con o sin parámetros), el compilador deja de generar el constructor por defecto.

Pregunta número 15

Which three options correctly describe the relationship between the classes?

```
class Class1 { String v1; }
class Class2 {
    Class1 c1;
    String v2;
}
class Class3 { Class2 c1; String v3; }
```

Seleccione una o más de una:

- [ ] a. Class2 has-a v3.
- [ ] b. Class1 has-a v2.
- [x] c. Class2 has-a v2.
- [x] d. Class3 has-a v1.
- [ ] e. Class2 has-a Class3.
- [x] f. Class2 has-a Class1.

Respuesta: En el diseño orientado a objetos en Java, la relación "has-a" se refiere a la composición o agregación, que ocurre cuando una clase declara una variable de instancia (un campo) que hace referencia a otro objeto o tipo de dato. Tenemos que analizar las clases presentadas: La primera opción correcta es la letra c, que nos dice que Class2 has-a v2. Esta opción es verdadera porque la clase Class2 declara explícitamente y de forma directa el campo String v2. La segunda opción correcta es la f, que nos dice que Class2 has-a Class1. Es verdadera porque la clase Class2 declara explícitamente el campo Class1 c1, estableciendo una relación directa de composición con Class1. También tenemos que el inciso d, que es Class3 has-a v1, es verdadero por la propiedad transitiva de la composición estructural de estas clases. Un objeto de Class3 contiene un objeto Class2 (mediante c1), el cual a su vez contiene un objeto Class1 (mediante su propio c1), el cual finalmente contiene la variable v1. Estructuralmente, Class3 "tiene un" v1 anidado en su estado (el cual sería accesible a través de c1.c1.v1 si los niveles de acceso lo permitieran).

Pregunta número 16

What is the result?

```
public class ScopeTest{
    int z;
    public static void main(String[] args){
        ScopeTest myScope = new ScopeTest();
        int z=6;
        System.out.print(z);
        myScope.doStuff();
        System.out.print(z);
        System.out.print(myScope.z);
    }
    void doStuff(){
        int z=5;
        doStuff2();
        System.out.print(z);
    }
    void doStuff2(){
        z=4;
    }
}
```

Seleccione una:

- a. 6565
- b. 6504
- c. 6550
- d. 6560
- e. 6564

Respuesta: El código nos muestra los conceptos de alcance (scope) y ocultamiento de variables (shadowing). Hay que seguir el flujo de ejecución para entender qué está pasando. Primero tenemos el main: Se instancia el objeto myScope, cuya variable de instancia z se inicializa por defecto en 0. En la siguiente línea, se declara una variable local z = 6 y se imprime directamente, mostrando el primer 6 en la

consola. De ahí nos vamos al doStuff(): Se llama a este método sobre la instancia creada. Aquí adentro se declara otra variable local z = 5 (la cual oculta temporalmente a la variable de instancia dentro de este bloque). Se invoca a doStuff2(). En doStuff2() no se declara ninguna variable local, por lo que la instrucción z = 4; apunta directamente a la variable de instancia del objeto, actualizando su valor en memoria de 0 a 4. Regresando a doStuff(), la instrucción System.out.print(z); imprime la variable local perteneciente a ese bloque, mostrando un 5. De vuelta en main, la siguiente línea es System.out.print(z), la cual imprime la variable local del método principal que se ha mantenido intacta, mostrando otro 6. Finalmente, System.out.print(myScope.z); accede de forma explícita a la variable de instancia del objeto. Como este valor fue reasignado a 4 durante la ejecución de doStuff2(), se imprime el 4.

Pregunta número 17

What is the result?

```
public class ScopeTest{
    int z;
    public static void main(String[] args){
        ScopeTest myScope new ScopeTest();
        int z = 6;
        System.out.println(z);
        myScope.doStuff();
        System.out.print(z);
        System.out.print(myScope.z);
    }
    void doStuff(){
        int z = 5;
        doStuff2();
        System.out.print(myScope.z);
    }
    void doStuff2(){
        z = 4;
    }
}
```

Seleccione una:

- a. Compilation fails
- b. 6564
- c. 6565
- d. 6554
- e. 6566

Respuesta: El código contiene dos errores de sintaxis y ámbito que impiden que el programa compile: Primero tenemos el error de sintaxis (falta el signo =). En la línea ScopeTest myScope new ScopeTest(); falta el operador de asignación. La sintaxis correcta debería ser: ScopeTest myScope = new ScopeTest(); ya que sin el =, el compilador de Java no reconoce esta línea como una declaración e inicialización válida. Lo segundo que debemos tomar en cuenta es el error de ámbito (variable no accesible): En el método doStuff(), la línea System.out.print(myScope.z); intenta

acceder a la variable myScope. Sin embargo, myScope fue declarada como una variable local dentro del método main. Las variables locales solo son visibles dentro del método donde se declaran. El método doStuff() no tiene conocimiento de la existencia de myScope, por lo que el compilador lanzará un error del tipo "cannot find symbol" (no se encuentra el símbolo).

Pregunta número 18

What is the result?

```
public class OverloadedExample {
    private String name;
    public OverloadedExample() {
        this("Default");
        System.out.println("No-argument constructor called");
    }
    public OverloadedExample(String name) {
        this.name = name;
        System.out.println("Constructor with argument called: " + name);
    }
    public void display() {
        System.out.println("Name: " + name);
    }
    public void display(String prefix) {
        System.out.println(prefix + ":" + name);
    }
    public static void main(String[] args) {
        OverloadedExample obj1 = new OverloadedExample();
        obj1.display();
        obj1.display("Hello");
        OverloadedExample obj2 = new OverloadedExample("John");
        obj2.display();
        obj2.display("Hi");
    }
}
```

Seleccione una:

- a. No-argument constructor called Constructor with argument called: Default Name: Default Hello:Default Constructor with argument called: John Name: John Hi:John
- b. No-argument constructor called Constructor with argument called: Default Name: Default Hello:Default Constructor with argument called: John Hi:John Name: John
- c. Constructor with argument called: Default No-argument constructor called Name: Default Hello:Default Constructor with argument called: John Hi:John Name: John
- d. Constructor with argument called: Default No-argument constructor called Name: Default Hello:Default Constructor with argument called: John Name: John Hi:John

Respuesta: En este código podemos evaluar el concepto de encadenamiento de constructores (constructor chaining) mediante el uso de this() y la sobrecarga de métodos. Veamos el flujo de ejecución para entender qué está pasando: Empecemos con el primer objeto (obj1): Al llamar a new OverloadedExample(), se invoca el constructor sin argumentos. La primera línea dentro de este es this("Default"), lo que pausa la ejecución actual y transfiere el control al constructor sobrecargado que recibe un String. Este constructor parametrizado asigna el valor a la variable de instancia y ejecuta su println, imprimiendo primero "Constructor with argument called: Default". Una vez finalizado, el control regresa al constructor sin argumentos para continuar con su bloque, imprimiendo "No-argument constructor called". Luego, las dos llamadas a los métodos sobrecargados display sobre obj1 imprimen "Name: Default" y "Hello:Default" de forma secuencial. Seguimos con el segundo objeto (obj2): La instrucción new OverloadedExample("John"); invoca directamente al constructor parametrizado, por lo que imprime únicamente "Constructor with argument called: John". Finalmente, las llamadas a los métodos sobre obj2 imprimen "Name: John" seguido de "Hi:John". La opción `d` plasma correctamente que el mensaje del constructor con argumentos aparece antes que el mensaje del constructor sin argumentos durante la creación del primer objeto, y mantiene el orden correcto de impresión para el resto de los métodos.

Pregunta número 19

What is the result?

```
public class OverloadedMethods {
    public OverloadedMethods() {
        this(10);
        System.out.println("No-argument constructor");
    }
    public OverloadedMethods(int x) {
        System.out.println("Constructor with int argument: " + x);
    }
    public void calculate() {
        System.out.println("No-argument calculate method");
    }
    public void calculate(int x) {
        System.out.println("Calculate method with int argument: " + x);
    }
    public static void main(String[] args) {
        OverloadedMethods obj1 = new OverloadedMethods();
        obj1.calculate();
        obj1.calculate(5);
        OverloadedMethods obj2 = new OverloadedMethods(20);
        obj2.calculate();
        obj2.calculate(15);
    }
}
```

Seleccione una:

- a. No-argument constructor Constructor with int argument: 10 No-argument calculate method Calculate method with int argument: 5 Constructor with int argument: 20 Calculate method with int argument: 15 No-argument calculate method
- b. No-argument constructor Constructor with int argument: 10 Calculate method with int argument: 5 No-argument calculate method Constructor with int argument: 20 No-argument calculate method Calculate method with int argument: 15
- c. Constructor with int argument: 10 No-argument constructor No-argument calculate method Calculate method with int argument: 5 Constructor with int argument: 20 No-argument calculate method Calculate method with int argument: 15
- d. Constructor with int argument: 10 No-argument constructor Calculate method with int argument: 5 Constructor with int argument: 20 No-argument calculate method Calculate method with int argument: 15 No-argument calculate method

Respuesta: Este ejercicio es muy parecido al anterior y vuelve a poner a prueba nuestro dominio sobre el encadenamiento de constructores mediante `this()` y la sobrecarga de métodos. Vamos a seguir la ejecución paso a paso dentro del método main. Empecemos por el primer objeto (obj1): La instrucción `new OverloadedMethods()` llama al constructor sin argumentos. La primera línea de este

constructor es `this(10)`, lo que desvía el flujo de ejecución hacia el constructor parametrizado. El constructor parametrizado imprime "Constructor with int argument: 10". El control regresa al constructor sin argumentos, el cual continúa su bloque e imprime "No-argument constructor". De ahí nos vamos a las llamadas a métodos de obj1: obj1.calculate() invoca el método sin parámetros, imprimiendo "No-argument calculate method". obj1.calculate(5) invoca el método sobrecargado que recibe un entero, imprimiendo "Calculate method with int argument: 5". Continuamos con el segundo objeto (obj2): La instrucción `new OverloadedMethods(20)` invoca directamente al constructor parametrizado, lo que imprime "Constructor with int argument: 20". Finalizamos con las llamadas a métodos de obj2: obj2.calculate() imprime "No-argument calculate method". obj2.calculate(15) imprime "Calculate method with int argument: 15". La opción c es la única que refleja con exactitud el orden correcto de esta secuencia de llamadas y salidas a consola.

Pregunta número 20

¿Cuál será el resultado del siguiente fragmento de código?

```
public class OverloadedConstructorsTest {
    private String name;
    private int age;
    public OverloadedConstructorsTest() {
        this("Unknown", 0);
        System.out.println("Default constructor called");
    }
    public OverloadedConstructorsTest(String name) {
        this(name, 0);
        System.out.println("Constructor with name called");
    }
    public OverloadedConstructorsTest(String name, int age) {
        this.name = name;
        this.age = age;
        System.out.println("Constructor with name and age called");
    }
    public static void main(String[] args) {
    OverloadedConstructorsTest obj1 = new OverloadedConstructorsTest();
    OverloadedConstructorsTest obj2 = new OverloadedConstructorsTest("Alice");
    OverloadedConstructorsTest obj3 = new OverloadedConstructorsTest("Bob", 30);
    }
}
```

Seleccione una:

a.

```
Default constructor called
Constructor with name and age called
Constructor with name called
Constructor with name and age called
Constructor with name and age called
```

b.

```
Constructor with name and age called
Default constructor called
Constructor with name called
Constructor with name and age called
```

c.

```
Constructor with name and age called
Default constructor called
Constructor with name and age called
Constructor with name called
Constructor with name and age called
```

d.

```
Default constructor called
Constructor with name called
Constructor with name and age called
```

Respuesta: El resultado de este ejercicio se debe al encadenamiento de constructores usando la palabra clave this(). Al llamar a otro constructor de la misma clase, la ejecución del constructor actual se pausa hasta que el constructor llamado termina su ejecución. Vamos a desglosar el flujo para cada objeto creado en el método main. Empecemos con el obj1 = new OverloadedConstructorsTest(): Se invoca el constructor sin argumentos. La instrucción this("Unknown", 0) delega inmediatamente la llamada al constructor que recibe (String, int). Este constructor final imprime la primera línea: "Constructor with name and age called". El control regresa al constructor sin argumentos, el cual termina e imprime: "Default constructor called". Seguimos con el obj2 = new OverloadedConstructorsTest("Alice"): Se invoca el constructor con un argumento

(String name). La instrucción `this(name, 0)` delega la llamada al constructor de dos argumentos (String, int). Este imprime: "Constructor with name and age called". El control regresa al constructor de un argumento, el cual imprime: "Constructor with name called". Terminamos con el obj3 = new OverloadedConstructorsTest("Bob", 30): Se invoca directamente el constructor de dos argumentos (String, int). Al no haber delegación `this()`, simplemente ejecuta su bloque e imprime: "Constructor with name and age called".

Pregunta número 21

Dado el siguiente código ¿cuál es el resultado?

```
public class Recursividad {
    public Recursividad() {
        this(10);
        System.out.println("Default");
    }
    public Recursividad(int val) {
        this();
        System.out.println("Numerico");
    }
    public static void main(String[] args) {
        new Recursividad();
    }
}
```

Seleccione una:

- a. Error de compilación
- b. Lanza StackOverflowError
- c. Imprime Numerico y luego Default
- d. Imprime Default y luego Numerico

Respuesta: Este ejercicio es realmente tramposo; si lo analizamos por encima, a simple vista la lógica nos sugiere que el programa entrará en un ciclo infinito y terminará lanzando un StackOverflowError en tiempo de ejecución (opción b), pero en realidad el programa ni siquiera llega a ejecutarse. Hay que partir de que el constructor sin argumentos Recursividad() invoca a this(10). El constructor parametrizado Recursividad(int val) invoca de regreso a this(). Esta estructura crea

un ciclo cerrado o dependencia circular entre los constructores. En Java, el compilador es lo suficientemente inteligente como para analizar y detectar ciclos en las llamadas a constructores mediante `this()`. Al detectar esta invocación recursiva a nivel estructural, el compilador interviene inmediatamente y lanza un error, impidiendo que el código se compile.

Pregunta número 22

What will be the result of attempting to compile and run the following code snippet?

```
public class RecursiveConstructorTest {
    private int value;

    public RecursiveConstructorTest() {
        this(10);
        System.out.println("No-argument constructor called");
    }

    public RecursiveConstructorTest(int value) {
        this();
        this.value = value;
        System.out.println("Constructor with int argument called: " + value);
    }

    public static void main(String[] args) {
        RecursiveConstructorTest obj = new RecursiveConstructorTest();
    }
}
```

Seleccione una:

- a. Compilation error due to recursive constructor calls.
- b. Runtime error due to infinite recursion.
- c. No-argument constructor called. Constructor with int argument called: 10.
- d. Constructor with int argument called: 10. No-argument constructor called.

Respuesta: Este ejercicio nos permite reforzar exactamente el mismo principio que vimos en la pregunta anterior. Tenemos una invocación recursiva circular entre los constructores: El constructor sin argumentos RecursiveConstructorTest() llama al constructor parametrizado mediante this(10). El constructor parametrizado RecursiveConstructorTest(int value) llama de regreso al constructor sin argumentos mediante this(). Entonces, como mencionamos anteriormente, el compilador de Java es capaz de detectar esta dependencia circular a nivel estructural. En lugar de compilar el código y permitir que falle en tiempo de ejecución con un error de

recursividad infinita (opción b); el compilador bloquea el proceso inmediatamente y lanza un error indicando que existe una invocación recursiva en los constructores.

Pregunta número 23

Give What is the result?

```
public class Test {
    public static void main(String[] args) {
        float f = 3.14f;
        new Test().doIt(f);
    }
    void doIt(Float f) {
        System.out.println("Float");
    }
    void doIt(double d) {
        System.out.println("double");
    }
    void doIt(float... fs) {
        System.out.println("varargs");
    }
}
```

Seleccione una:

- a. Compilations fails.
- b. An exception is thrown at runtime.
- c. The output is not predictable.
- d. Float
- e. varargs
- f. double

Respuesta: En este ejercicio se están evaluando las reglas de prioridad del compilador de Java para resolver la sobrecarga de métodos (method overloading resolution). Cuando el método main invoca doIt(f) pasando la variable f de tipo primitivo float, el compilador busca el método que mejor coincida siguiendo este orden estricto. Primero va por la coincidencia exacta: El compilador busca primero un método doIt(float). Al no encontrarlo, avanza a la siguiente regla, que es la

promoción de tipos primitivos (widening): Busca un método que acepte un primitivo de mayor capacidad. En este escenario que tenemos, se encuentra doIt(double d), ya que un float puede promoverse de forma segura a un double. De ahí seguimos con el autoboxing: Si la promoción no fuera posible, el compilador intentaría envolver el primitivo en su clase wrapper (Float), lo que llamaría a doIt(Float f). Finalmente, llegamos con varargs: Como último recurso de resolución, el compilador de Java utilizaría el método con argumentos de longitud variable doIt(float... fs). Debido a que la promoción de tipos primitivos (widening) tiene mayor precedencia en Java que el autoboxing y que los varargs, el compilador enlaza la llamada directamente al método void doIt(double d).

Pregunta número 24

¿Cuántos de los fragmentos de código, si compilan?

```
public static void main(String...a) {
public static void main(String. a) {
public static void main(String... a) {
public static void main(String[]... a) {
public static void main(String...[] a) {
```

Seleccione una:

- a. 2
- b. 3
- c. Todos
- d. Ninguno

Respuesta: Para determinar cuántos fragmentos compilan, debemos analizar la sintaxis de cada uno según las reglas de Java, especialmente en lo que respecta a los argumentos variables (varargs). Empecemos con el primer fragmento (public static void main(String...a) {). Este fragmento sí compila, ya que es la sintaxis estándar de varargs. El tipo es String, seguido de ... y el nombre de la variable `a`. El espacio entre ... y `a` no es obligatorio. Vamos con el siguiente fragmento: (public static void main(String. a) {). Este no compila, ya que hay un error de sintaxis. Primero que nada, `String.` no es un tipo válido. El punto (.) es un carácter inválido en este contexto. El tercer fragmento (public static void main(String... a) {) sí compila. Es idéntico al primer caso, solo que con un espacio entre ... y `a`. El compilador de Java ignora los espacios en blanco, por lo que es válido. El cuarto fragmento (public static void main(String[]... a) {) sí compila. Esta es una sintaxis

válida para un número variable de argumentos que son arreglos de Strings. El tipo es String[] y se le aplica varargs con `...`. Internamente, `a` se trata como un String[][]. Finalmente, el quinto fragmento (public static void main(String...[] a) {) no compila. Esto se debe a que la sintaxis de varargs (...) debe ir inmediatamente después del tipo de dato. No se pueden colocar [] después de los puntos suspensivos. El compilador espera el nombre de la variable después de ..., no corchetes. Haciendo un recuento, tenemos que los fragmentos que compilan son el 1, 3 y 4, dando un total de 3 fragmentos que sí compilan.